

Report

Applying Data Structures in the Development of Aetheris

(Project SERAAC)

Prepared by

MTG (MorThodGame)

1. [Full name] Student ID [.....]
2. [Full name] Student ID [.....]
3. [Full name] Student ID [.....]
4. [Full name] Student ID [.....]

Presented to

[Instructor's name]

This report is part of [Course code] Data Structures

Semester [..], Academic Year 2026

Chiang Mai University

Preface

This report is part of the Data Structures course. It describes the data structures our group uses, and plans to use, in Aetheris (project name SERAAC), an RPG in which the player takes a party of four into a dungeon, fights turn-based battles, collects loot and decides whether to extract or keep going. The game is written in C# on .NET 9 with MonoGame 3.8.

The report has five chapters: introduction, related theory, system design, how the data structures are used in the game, and conclusion. All code samples and diagrams come from the project's source. The appendices contain more code and a list of the data structures used.

We hope the report is useful to its readers, and apologise for any mistakes it may contain.

The authors
MTG

Table of Contents

Page

Preface **2**

Table of Contents **3**

List of Figures **5**

List of Tables **6**

Chapter 1 Introduction **7**

1.1 Background 7

1.2 Objectives 7

1.3 Scope 7

1.4 Overview of Aetheris 7

1.5 Expected Benefits 8

1.6 Definitions 8

Chapter 2 Related Theory **9**

2.1 Performance Analysis 9

2.2 Arrays and Lists 9

2.3 Stacks and Queues 9

2.4 Hash Tables 10

2.5 Graphs, Trees and Breadth-First Search 10

2.6 Bitmasks and Multi-key Sorting 10

Chapter 3 System Design **11**

3.1 Program Structure 11

3.2 Selection Criteria 11

3.3 Class Diagram 12

Chapter 4 Data Structures in the Game **14**

4.1 The Screen Stack 14

4.2 The Dungeon Map as a Graph 15

4.3 Room Layouts as 2D Arrays 17
4.4 Pathfinding with BFS 18
4.5 The Formation and the Position Bitmask 20
4.6 Turn Order and the Event Queue 22
4.7 Status Effects and Enemy AI 23
4.8 Content Data and the Dialogue Graph 24
4.9 Asset Caches and Flags 25
4.10 Saving 26
4.11 Planned Data Structures 26

Chapter 5 Conclusion 27
5.1 Summary 27
5.2 Problems Encountered 27
5.3 Recommendations 28

References 29

Appendix A Map Generation Code 30

Appendix B Data Structures in the Project 32

Appendix C Sample Save File 33

List of Figures

Page

Figure 3.1 Program structure and the main data structures in each part 11

Figure 3.2 UML class diagram of the main classes 13

Figure 4.1 Pushing and popping the combat screen 14

Figure 4.2 The room hash table and the adjacency list inside a room 15

Figure 4.3 An example Trial map: solid lines are tree edges, dashed lines are loops, d is the BFS depth 16

Figure 4.4 The queue, the predecessor table and path reconstruction 18

Figure 4.5 The Plaza layout in Tile[21,12] with BFS distances and the shortest path 20

Figure 4.6 The formation arrays, a swap, and an empty slot after a death 20

Figure 4.7 A set of positions stored as a bitmask 21

Figure 4.8 The order of work in one turn 23

Figure 4.9 Weighted random target selection 24

Figure 4.10 The dialogue graph of the NPC Maren 25

Figure 4.11 Caching assets in a Dictionary 25

List of Tables

Page

Table 1.1 Terms used in this report 8
Table 2.1 Time complexity of the main operations (average case) 10
Table 3.1 Game systems and the data structures they use 11
Table 4.1 Data structures planned for future work 26
Table 5.1 Summary of the data structures used in the game 27
Table B.1 Fields in the code that are data structures 32

Chapter 1
Introduction

1.1 Background

Games run in real time. Aetheris renders at 60 frames per second, so each frame has about 16.7 ms to read input, update characters and enemies, and draw everything. A poor choice of data structure, such as a linear search through a long list every frame, can easily make the game stutter.

A data structure that matches the game's rules also makes the code easier to read and less buggy. For example, the design document says a formation has 4 slots and a slot stays empty when its unit dies. An array of 4 with `null` for empty slots matches that rule directly.

1.2 Objectives

1. To describe the data structures used in Aetheris, with diagrams and code.
2. To explain why each system uses the data structure it does.
3. To analyse the time complexity of the main operations.
4. To present the data structures planned for future work.

1.3 Scope

The report covers the SERAAC source on the `dev` branch, about 60 C# files, excluding the old code in `_Archive`. It covers screens, the dungeon map, room layouts, pathfinding, combat, enemy AI, dialogue, asset loading and saving.

1.4 Overview of Aetheris

The player starts in the Hub, picks 4 characters, sets their formation, equips them and upgrades their stats. They then enter a Trial, a dungeon of connected rooms generated anew each time. Rooms may contain enemies,

chests, rest spots, events or extraction points. Walking into an enemy starts a battle: the player chooses an action for every party member, then both sides' actions run in order of speed. Loot found in a Trial only becomes the player's when they extract; if the party is defeated, it is all lost.

1.5 Expected Benefits

1. An understanding of how data structures are used in real-time programs.
2. The ability to compare data structures before choosing one.
3. A reference for group members as development continues.

1.6 Definitions

Table 1.1 Terms used in this report

Term	Meaning
Trial	One dungeon, made of connected rooms
Room	One room in a Trial, with a type, a layout, connections and a state
Slot	One of 4 formation positions: Front 1, Front 2, Back 1, Back 2
Combatant	One character or enemy in a battle
Extraction	Leaving a Trial to keep the loot
Screen	One game state, e.g. menu, hub, exploration, combat

Chapter 2
Related Theory

2.1 Performance Analysis

Performance is described with Big-O notation, which says how running time grows with the input size n: O(1) is constant, O(n) grows linearly, and O(n log n) is the best a comparison sort can do. Some costs are amortized: appending to a List occasionally grows the underlying array, but costs O(1) on average. Games also have to watch memory allocation, because the .NET garbage collector can cause a visible hitch if too many short-lived objects are created.

2.2 Arrays and Lists

An array (`T[]`) stores elements contiguously and gives O(1) access by index, but its size is fixed. A 2D array (`T[,]`) suits grid-shaped data, and a jagged array (`T[][]`) is an array of arrays whose rows may have different lengths.

`List<T>` is an array that grows by itself. Appending is amortized O(1) and indexing is O(1), but inserting or removing in the middle and searching by value are O(n). It suits data that is appended and then iterated.

2.3 Stacks and Queues

A stack is LIFO (last in, first out), with O(1) Push and Pop. It fits work that nests and then unwinds, such as undo or screens opened on top of each other. A queue is FIFO (first in, first out), with O(1) Enqueue and Dequeue. .NET's `Queue<T>` is a circular buffer, so nothing is shifted. Queues fit work that must be handled in arrival order.

2.4 Hash Tables

A hash table stores key–value pairs and uses the key's hash to find where an entry lives, so insert, lookup and delete are O(1) on average. In C# this is `Dictionary<K,V>`. `HashSet<T>` stores keys only; it tests membership in O(1) and holds no duplicates. Keys can be strings, enums, structs such as `Point`, or tuples.

2.5 Graphs, Trees and Breadth-First Search

A graph has vertices and edges. It can be stored as an adjacency matrix ($O(V^2)$ space) or as adjacency lists ($O(V + E)$ space), which suit graphs with few edges. A tree is a connected graph with no cycles.

Breadth-first search (BFS) starts at a source and expands one layer at a time using a queue. When every edge has the same cost, BFS finds the shortest distance to every vertex in $O(V + E)$. Recording each vertex's predecessor lets you rebuild the path. Using BFS on a grid to find every reachable cell is called flood fill.

2.6 Bitmasks and Multi-key Sorting

A bitmask uses the bits of an integer to represent a small set. Membership, union and subset tests are single bitwise operations, O(1). A multi-key sort orders by a primary key and uses secondary keys to break ties. In C# this is `OrderBy(...).ThenBy(...)`, a stable sort (equal elements keep their order) that runs in $O(n \log n)$.

Table 2.1 Time complexity of the main operations (average case)

Structure	Access	Search by value	Insert	Delete
`T[]`, `T[,]`	O(1)	O(n)	-	-
`List<T>`	O(1)	O(n)	O(1) at end	O(n)
Stack / Queue	top/front O(1)	O(n)	O(1)	O(1)
`Dictionary` / `HashSet`	O(1)	by key O(1)	O(1)	O(1)
Bitmask	O(1)	O(1)	O(1)	O(1)

Chapter 3
System Design

3.1 Program Structure

The code is split by responsibility. `Game1` is MonoGame's main loop, calling `Update()` and `Draw()` every frame. `ScreenManager` keeps the screens as a stack. The game rules live outside the screens: Trial (dungeons and rooms), Combat (battle rules with no drawing code), Meta (saved data) and Data (game content). This split lets each part use the data structure that suits its own work.

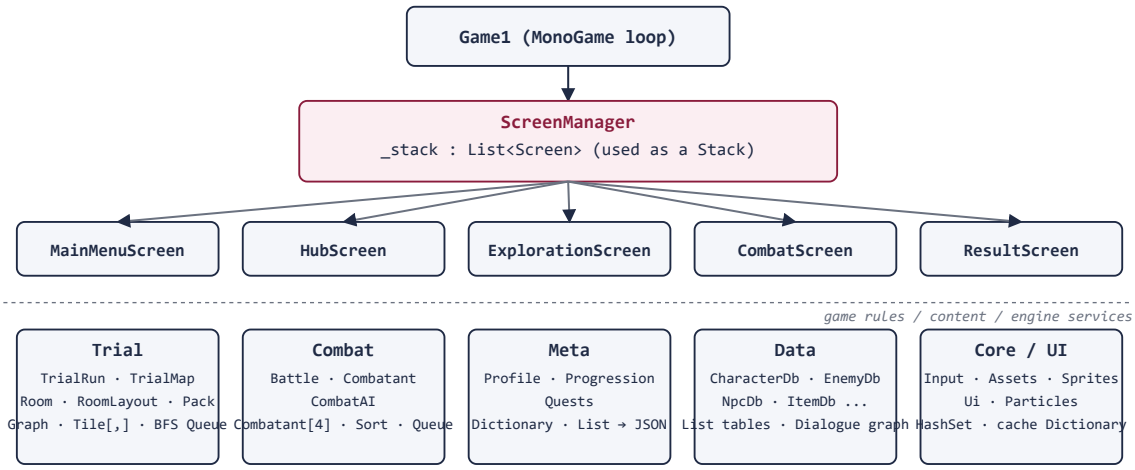


Figure 3.1 Program structure and the main data structures in each part

3.2 Selection Criteria

We considered four things: (1) how the system accesses its data, e.g. by index, by key or in arrival order; (2) whether the data is fixed or variable in size, and how large it is (for ten items or fewer, a linear search is fine); (3) whether the code runs every frame or only once; and (4) whether the structure matches the rule in the design document.

Table 3.1 Game systems and the data structures they use

System	Data structure	Section

Screen management	Stack (`List<Screen>`)	4.1
Trial map	Graph: `Dictionary<Point,Room>` and `Dictionary<Dir,Room>`, BFS	4.2
Room layout	`Tile[,]`, `string[][]`, `HashSet<Point>`	4.3
Pathfinding	BFS with `Queue` and `Dictionary`	4.4
Formation	`Combatant[4]`, `Ranks` bitmask	4.5
Turn order	Multi-key sort, two `Queue`s	4.6
Statuses and AI	`List<StatusInstance>`, `List<(T, double)>`	4.7
Content and dialogue	`List`, directed graph	4.8
Caches and flags	`Dictionary`, `HashSet`	4.9
Saving	`Dictionary` and `List` as JSON	4.10

3.3 Class Diagram

The UML diagram below shows the main classes and their data-structure fields. A filled diamond marks composition (one object owns another), a dashed line marks a dependency, and the triangle marks inheritance.

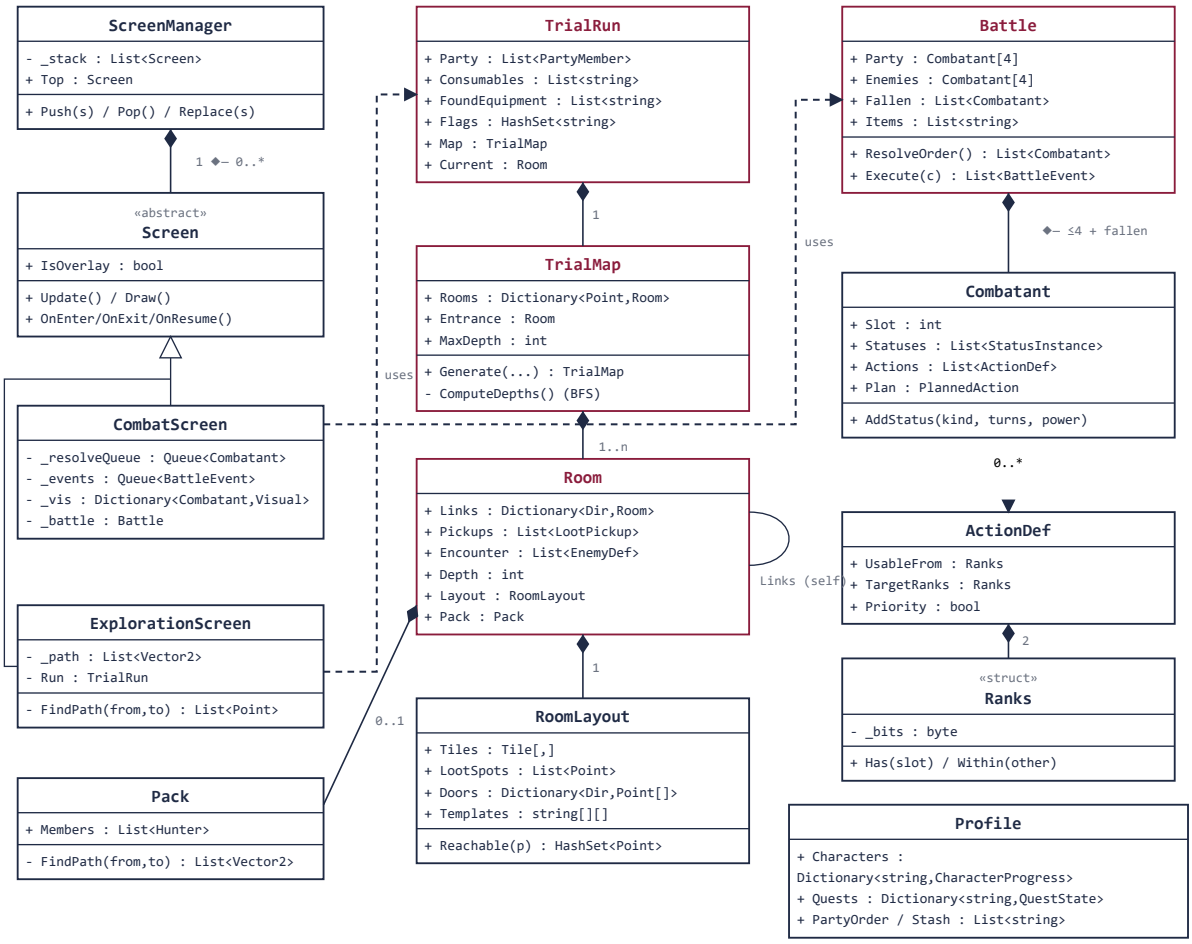


Figure 3.2 UML class diagram of the main classes

Chapter 4
Data Structures in the Game

4.1 The Screen Stack

The game has several screens: menu, hub, exploration, combat and results. When a fight starts during exploration, the exploration screen must be paused as it is and restored once the fight ends. `ScreenManager` therefore keeps screens in a `List<Screen>` used as a stack, with the end of the list as the top. Combat is pushed on top of exploration and popped when it ends. The exploration screen stays in memory, so nothing has to be rebuilt.

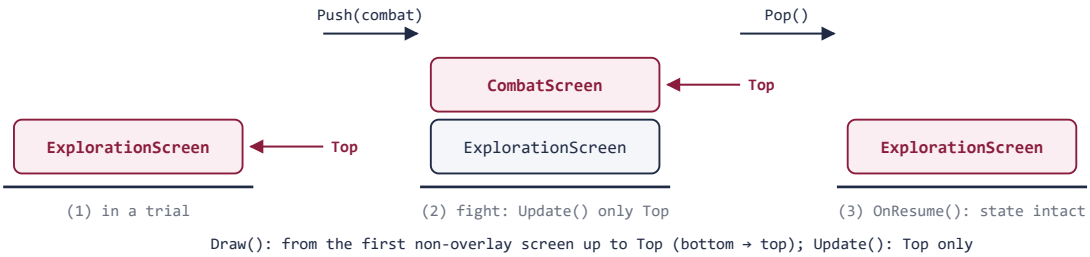


Figure 4.1 Pushing and popping the combat screen

```
public void PopInstant()
{
    if (_stack.Count > 0)
    {
        _stack[^1].OnExit();
        _stack.RemoveAt(_stack.Count - 1);   // remove the top: O(1)
    }
    Top?.OnResume();
}

public void Draw(SpriteBatch sb, GameTime gt)
{
    for (int i = FirstVisible(); i < _stack.Count; i++)   // bottom -> top
        _stack[i].Draw(sb, gt);
}
```

Listing 4.1 Popping the top screen and drawing bottom to top (Core/Screen.cs)

A List is used instead of `Stack<T>` because the pause menu is translucent, so the screen beneath it must also be drawn. That means iterating bottom to top by index, which `Stack<T>` does not support. Push and Pop are O(1).

4.2 The Dungeon Map as a Graph

Each Trial needs a new random map. Every room must be reachable, there should be branches and loops, and special rooms must be placed by distance from the entrance: the objective at the deepest dead end, and extraction points reasonably far from the entrance.

The map is an undirected graph stored at two levels. The first is `Dictionary<Point, Room> Rooms`, keyed by grid coordinates, which finds a room by position or checks whether a cell is taken in O(1). The second is `Dictionary<Dir, Room> Links` in each room, an adjacency list keyed by direction. A room has at most 4 links, and the key also tells which side the door is on.

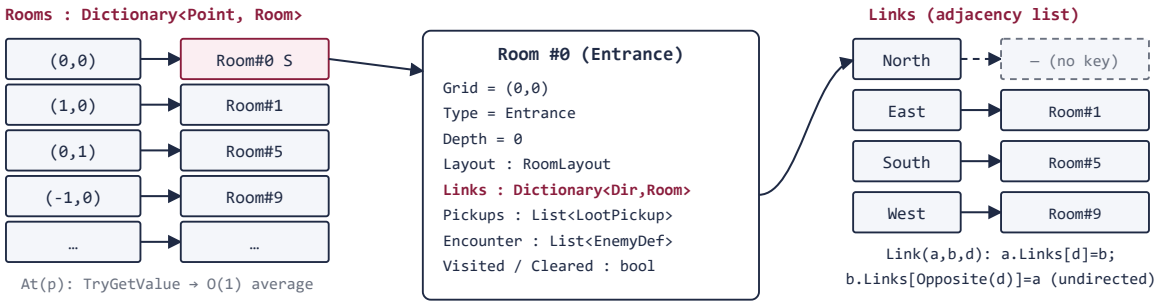


Figure 4.2 The room hash table and the adjacency list inside a room

Generation starts with the entrance at (0,0) and grows new rooms out of existing ones, one at a time. Each new room is linked to exactly one existing room, so the result is a tree in which every room is reachable. Neighbouring rooms are then linked with an 18% chance to create loops. Finally BFS computes each room's depth (the number of doors from the entrance), which decides room types and difficulty.

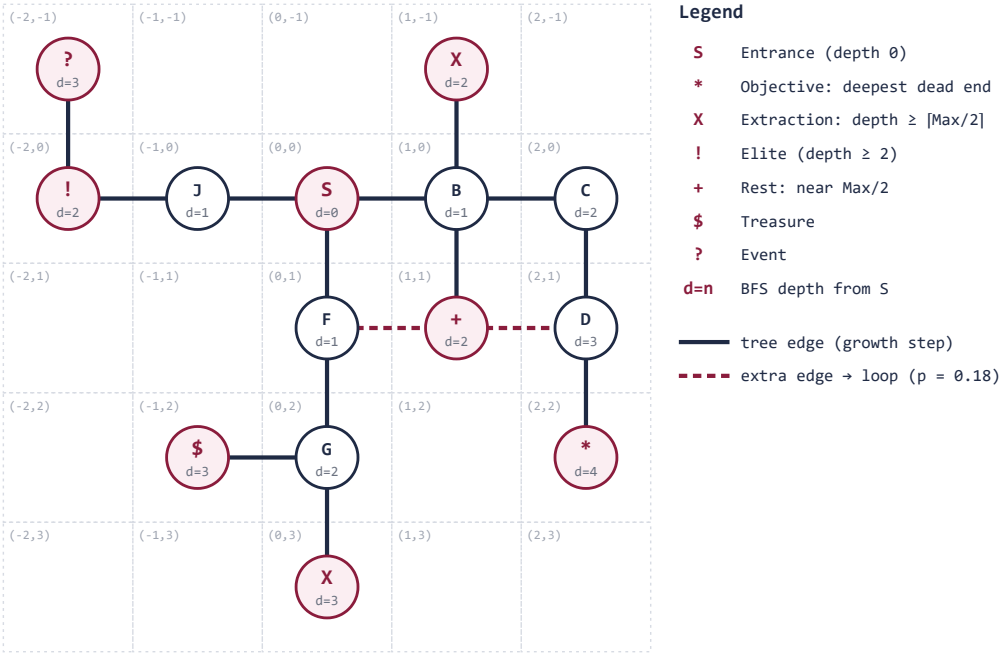


Figure 4.3 An example Trial map: solid lines are tree edges, dashed lines are loops, d is the BFS depth

```
private static void Link(Room a, Room b, Dir aToB)
{
    a.Links[aToB] = b;                          // undirected edge = two entries
    b.Links[RoomLayout.Opposite(aToB)] = a;
}

private void ComputeDepths()
{
    foreach (var r in Rooms.Values) r.Depth = -1;   // -1 = not visited
    var q = new Queue<Room>();
    Entrance.Depth = 0;
    q.Enqueue(Entrance);
    while (q.Count > 0)
    {
        var r = q.Dequeue();
        foreach (var n in r.Links.Values)
            if (n.Depth < 0) { n.Depth = r.Depth + 1; q.Enqueue(n); }
    }
    MaxDepth = Rooms.Values.Max(r => r.Depth);
}
```

Listing 4.2 Linking rooms both ways and computing depths with BFS (Trial/TrialMap.cs)

A `Room[9,9]` array was not used because a map has only 8–15 rooms out of 81 cells, and coordinates can be negative; the Dictionary stores only rooms that exist. BFS is used instead of DFS because we need the shortest distance

from the entrance, which DFS does not guarantee once there are loops. BFS runs in O(V + E), once per Trial.

4.3 Room Layouts as 2D Arrays

Each room is divided into 48-pixel tiles. The game needs to know which tiles are walkable, which are walls or pillars, and where loot can be placed. Collision checks run every frame for the party and every enemy, so they must be fast.

Five template layouts are stored as `string[][] Templates`, one character per tile (`.` floor, `o` pillar, `L` loot spot, `F` feature), so layouts can be edited by typing. When a room is built, its template is converted into a `Tile[21,12]` (optionally mirrored left to right) and doors are opened for each direction in `Links`.

```
for (int iy = 0; iy < InnerRows; iy++)
    for (int ix = 0; ix < InnerCols; ix++)
    {
        char ch = src[iy][mirror ? InnerCols - 1 - ix : ix];   // mirror = read the
row backwards
        int x = ix + 1, y = iy + 1;                            // +1: outer wall ring
        layout.Tiles[x, y] = ch switch { 'o' => Tile.Pillar, ',' => Tile.Decor, _ =>
Tile.Floor };
        if (ch == 'L') layout.LootSpots.Add(new Point(x, y));
    }

public bool Solid(int x, int y) =>
    x < 0 || y < 0 || x >= Cols || y >= Rows || Tiles[x, y] == Tile.Wall || Tiles[x,
y] == Tile.Pillar;
```

Listing 4.3 Converting a template to a 2D array, and the solid-tile test (Trial/RoomLayout.cs)

Rooms are a fixed size and every tile holds data, so a 2D array is the natural fit, and `Solid(x, y)` is O(1). The `Reachable()` method also runs BFS with a `HashSet<Point>` to find every tile reachable from a door. The game uses this to choose enemy spawn points (`Pack.Spawn`) and to move dropped loot onto reachable floor (`TrialRun.ClampToFloor`).

4.4 Pathfinding with BFS

The player can click anywhere in a room to walk there, and chasing enemies periodically re-plan their path to the player. Both use BFS over the room grid: a `Queue<Point>` holds tiles waiting to be explored, and a `Dictionary<Point, Point> prev` holds each tile's predecessor and doubles as the visited set. Once the goal is reached, the path is rebuilt by following `prev` back to the start and reversing the list.

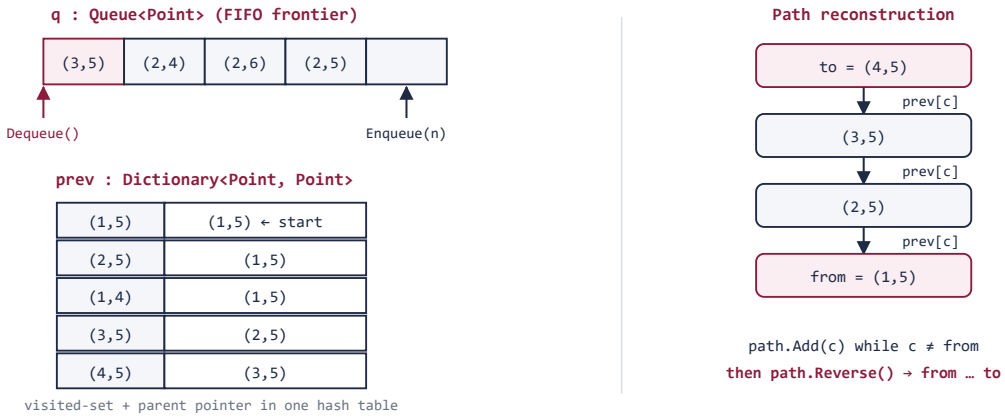


Figure 4.4 The queue, the predecessor table and path reconstruction

```
private List<Point> FindPath(Point from, Point to)
{
    if (from == to) return new List<Point> { to };
    var prev = new Dictionary<Point, Point> { [from] = from };
    var q = new Queue<Point>();
    q.Enqueue(from);
    while (q.Count > 0)
    {
        var p = q.Dequeue();
        if (p == to) break;                                  // shortest path found
        foreach (var d in RoomLayout.AllDirs)
        {
            var s = RoomLayout.Step(d);
            var n = new Point(p.X + s.X, p.Y + s.Y);
            if (prev.ContainsKey(n) || !Walkable(n)) continue;
            prev[n] = p;
            q.Enqueue(n);
        }
    }
    if (!prev.ContainsKey(to)) return null;                  // unreachable
    var path = new List<Point>();
    for (var c = to; c != from; c = prev[c]) path.Add(c);
    path.Reverse();
    return path;
}
```

Listing 4.4 Pathfinding with BFS (Screens/ExplorationScreen.cs)

Figure 4.5 shows the result on the real Plaza layout. Each number is the BFS distance from the left door; 186 tiles are reachable, and the path to the tile in front of the feature is 15 steps. After the path is found, waypoints that can be reached in a straight line are skipped, so the character cuts corners instead of zig-zagging.

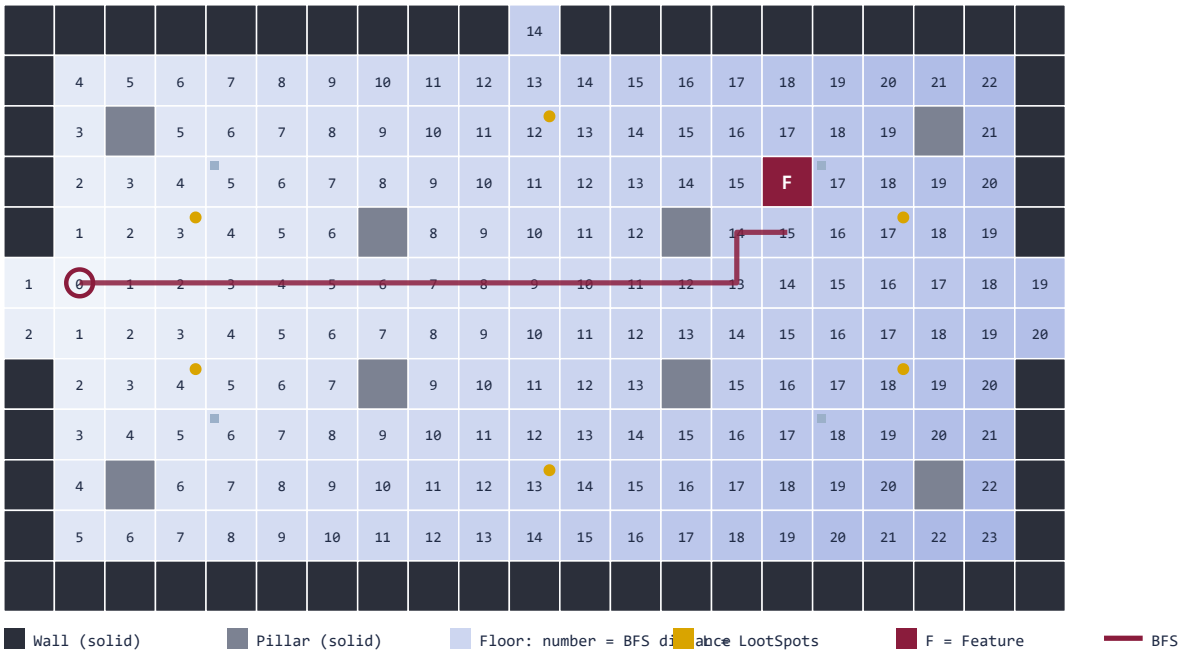


Figure 4.5 The Plaza layout in Tile[21,12] with BFS distances and the shortest path

We chose BFS over A* because a room has only 252 tiles and every step costs the same, so BFS finds an equally short path with simpler code and no priority queue. The cost is O(W × H) for a room of W × H tiles.

4.5 The Formation and the Position Bitmask

Per the design document, each side has 4 positions, characters can swap positions, and when a unit dies its position stays empty instead of someone moving up. `Battle` therefore uses a `Combatant[4]` array per side, where the index is the position (0 = Front 1 to 3 = Back 2) and an empty slot is `null`. Dead units move to `List<Combatant> Fallen`.

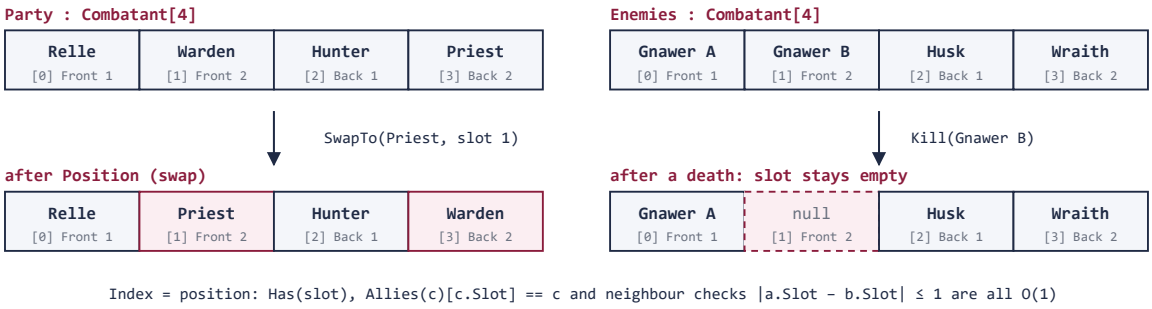


Figure 4.6 The formation arrays, a swap, and an empty slot after a death

```
private void SwapTo(Combatant c, int slot, List<BattleEvent> ev)
{
    var side = Allies(c);
    int from = c.Slot;
    var other = side[slot];        // may be null (empty slot)
    side[slot] = c;
    side[from] = other;
    c.Slot = slot;
    if (other != null) other.Slot = from;
}

private void Kill(Combatant t, List<BattleEvent> ev)
{
    var side = Allies(t);
    if (side[t.Slot] == t) side[t.Slot] = null;   // the position becomes empty
    Fallen.Add(t);
}
```

Listing 4.5 Swapping and dying (Combat/Battle.cs)

With a List, removing a dead unit would shift everyone behind it forward, which breaks the rule. An array with `null` slots matches the rule, and finding who is in a slot or swapping two slots is O(1). The downside is that every loop has to check for `null`.

Each skill lists the positions it can be used from and the positions it can hit. These are stored in `Ranks`, a struct holding a single `byte`, where bits 0–3 are the four positions. Membership is an AND with `1 << slot`, and a subset test is `(a & ~b) == 0`. Every operation is O(1) and allocates nothing.

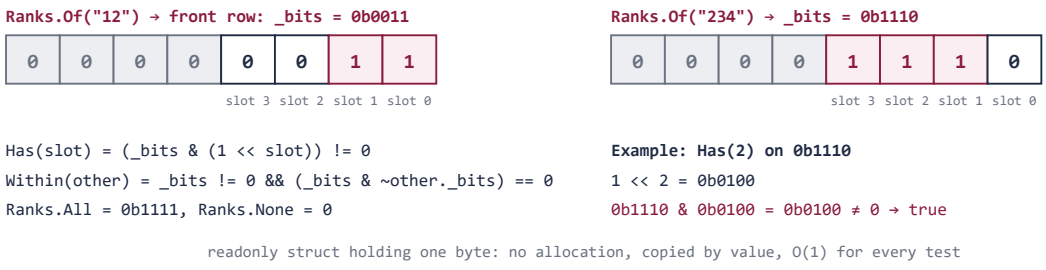


Figure 4.7 A set of positions stored as a bitmask

```
public readonly struct Ranks
{
    private readonly byte _bits;

    public static Ranks Of(string ranks)              // "12" -> 0b0011
    {
        byte b = 0;
        foreach (char ch in ranks)
            if (ch >= '1' && ch <= '4') b |= (byte)(1 << (ch - '1'));
        return new Ranks(b);
    }

    public bool Has(int slot) => slot >= 0 && slot < 4 && (_bits & (1 << slot)) != 0;
    public bool Within(Ranks other) => _bits != 0 && (_bits & ~other._bits) == 0;
}
```

Listing 4.6 The Ranks struct (Data/ActionDef.cs)

4.6 Turn Order and the Event Queue

Each turn the player chooses an action for every member (with undo). Then all actions run in order: Priority actions first, then by speed from high to low, with ties going to the party and then to front slots. The rules resolve instantly, but the results have to be shown step by step.

While choosing, the game uses a `List<Combatant>` with a cursor; undo decrements the cursor, which behaves like a stack. When every choice is made, `ResolveOrder()` sorts by several keys and the result goes into a `Queue<Combatant>`. Actors are dequeued one at a time, and `Battle.Execute()` returns a list of events (damage, moves, deaths) that go into a second `Queue<BattleEvent>` to be animated one by one.

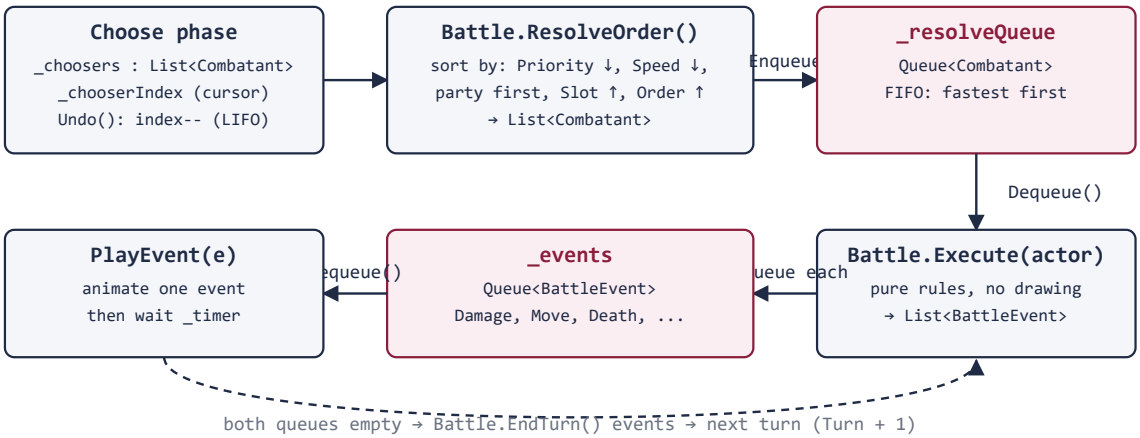


Figure 4.8 The order of work in one turn

```
public List<Combatant> ResolveOrder() =>
    Living.Where(c => c.Plan != null)
        .OrderByDescending(c => c.Plan.Action.Priority)
        .ThenByDescending(c => c.EffectiveSpeed)
        .ThenBy(c => c.IsParty ? 0 : 1)
        .ThenBy(c => c.Slot)
        .ThenBy(c => c.Order)          // spawn order: no ties left
        .ToList();

// CombatScreen
if (_events.Count > 0) _timer = PlayEvent(_events.Dequeue());
else if (_resolveQueue.Count > 0)
{
    var actor = _resolveQueue.Dequeue();
    foreach (var e in _battle.Execute(actor)) _events.Enqueue(e);
}
```

Listing 4.7 Multi-key sort and the two queues (Combat/Battle.cs, Screens/CombatScreen.cs)

A battle has at most 8 units, so one O(n log n) sort is enough, and the same order feeds the turn-order bar on screen. A stable sort with a unique last key gives the same order every time. The two queues keep the rules separate from the animation and show events in the order they happened.

4.7 Status Effects and Enemy AI

Each unit keeps its status effects (e.g. Guard, Mark, Slow) in a `List<StatusInstance>`. If it gains a status it already has, the game merges it into the existing entry, keeping the larger values, instead of adding a duplicate.

There are only 8 kinds of status, so a linear search is fast enough, and `RemoveAll()` removes statuses by condition in one pass.

Enemies choose targets by weighted random selection (a roulette wheel). They build a `List<(Combatant, double)>` of targets and weights: wounded or Marked targets weigh more, and each enemy type multiplies by its preference for each position (a `float[4]`). A random number between 0 and the total weight is drawn, and weights are subtracted one by one until it reaches zero. This is O(n), which is fine for at most 4 targets.

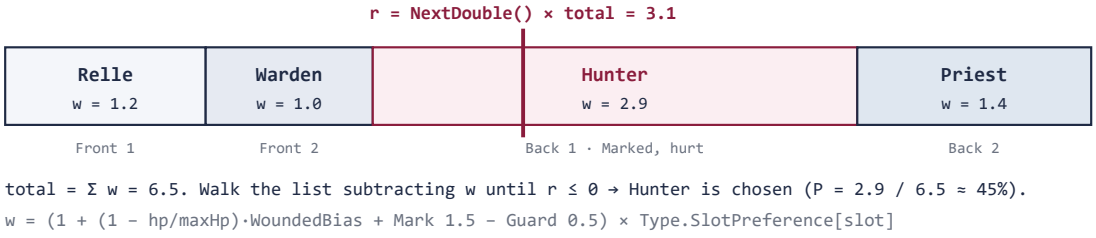


Figure 4.9 Weighted random target selection

4.8 Content Data and the Dialogue Graph

Characters, enemies, items and dungeons are stored as `List`s in classes such as `CharacterDb`, `EnemyDb` and `ItemDb`, because they are usually read in full (to show a menu or pick enemies) and the list order is the display order.

NPC dialogue is a directed graph stored in a `Dictionary<string, DialogueNode>`. Each node has text and a list of choices; each choice is an edge to the next node by name and can be hidden by a condition. Referring to nodes by name makes it easy to loop back to an earlier node, and finding the next node is O(1).

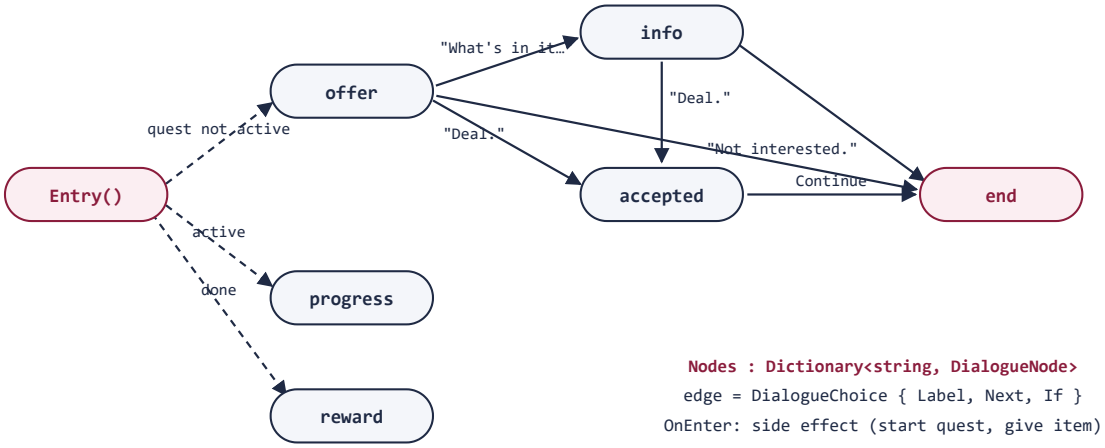


Figure 4.10 The dialogue graph of the NPC Maren

4.9 Asset Caches and Flags

Loading an image, cutting frames, scaling or recolouring a sprite takes time, so the game stores the results in a `Dictionary`: the first call loads and stores, and later calls read straight from the table. Keys vary: a file name (`string`), an image height (`int`), or a `(string, uint)` tuple for recoloured sprites, which avoids building string keys.

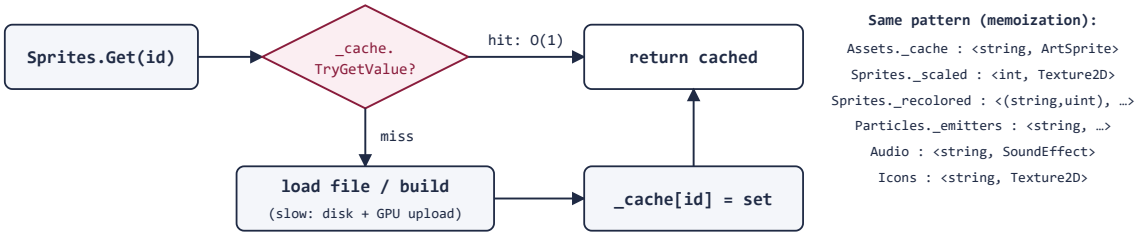


Figure 4.11 Caching assets in a Dictionary

For "has this happened yet?" data, the game uses `HashSet`. `TrialRun.Flags` records events from the current Trial for use as dialogue conditions, and the combat screen keeps the units currently being aimed at in a `HashSet<Combatant>`, which is checked for every unit every frame.

4.10 Saving

Data that persists between Trials lives in the `Profile` class and is saved as JSON. Each character's upgrade levels and equipment are stored in a `Dictionary<string, CharacterProgress>` keyed by character id, so old saves still work when new characters are added. The party order and the equipment stash are `List<string>`, because order matters. Both types map directly to JSON (see Appendix C). Loot found during a Trial is kept separately in `TrialRun` and only moves into `Profile` after a successful extraction.

4.11 Planned Data Structures

Table 4.1 Data structures planned for future work

Data structure	Use and reason
`PriorityQueue` and A*	If slow terrain (e.g. mud) is added, the grid becomes weighted and BFS no longer finds the best path, so pathfinding would switch to A*.
Object pool (`Stack<T>`)	Reuse short-lived objects such as damage numbers to reduce garbage collection.
Ring buffer	The combat log currently uses `List.RemoveAt(0)`, which is O(n); a ring buffer does this in O(1).
Spatial hash	If rooms hold many enemies, dividing the room into cells limits collision checks to nearby units.
DAG and topological sort	Unlock requirements for areas and quests, to find a valid order and check that no requirements form a cycle.

Chapter 5
Conclusion

5.1 Summary

Aetheris uses every group of basic data structures from the course, each chosen for how its system uses data, as summarised in Table 5.1. Systems that run every frame, such as collision and aiming, use O(1) operations, while heavier work such as BFS and map generation runs occasionally on small data.

Table 5.1 Summary of the data structures used in the game

Data structure	Use in the game
Array / 2D array	4-slot formation, 21×12 room grid
List	Content data, status effects, loot
Stack	Screens, undo while choosing actions
Queue	BFS, actor order, event order
Dictionary / HashSet	Rooms by position, adjacency, caches, dialogue, flags, save file
Graph / tree	Trial map, dialogue graph
Bitmask	Positions a skill can be used from and hit

5.2 Problems Encountered

1. Using `null` for empty formation slots means every loop must check for null, so we put the filtering into properties such as `LivingParty`.
2. LINQ calls such as `Where` and `OrderBy` allocate new objects each time. That is fine in occasional code but needs care in per-frame code.
3. .NET does not guarantee the enumeration order of `Dictionary`, so map generation, which must give the same result for the same seed, avoids removing entries while it runs.

5.3 Recommendations

For future development, the group should switch to A* once terrain with different movement costs is added, pool short-lived combat objects, and index the content tables with a Dictionary as the amount of content grows.

References

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to Algorithms* (4th ed.). MIT Press.

Microsoft. (2025). *System.Collections.Generic Namespace*. Microsoft Learn. https://learn.microsoft.com/dotnet/api/system.collections.generic

MonoGame Foundation. (2024). *MonoGame Documentation*. https://docs.monogame.net

Nystrom, R. (2014). *Game Programming Patterns*. Genever Benning. https://gameprogrammingpatterns.com

Patel, A. (2014). *Introduction to the A* Algorithm*. Red Blob Games. https://www.redblobgames.com/pathfinding/a-star/introduction.html

Appendix A

Map Generation Code

```
public static TrialMap Generate(TrialDef trial, Random rng, bool withNpcRoom = false,
int objectiveRooms = 0)
{
    var map = new TrialMap();
    var start = new Room { Id = 0, Grid = Point.Zero, Type = RoomType.Entrance };
    map.Rooms[start.Grid] = start;
    map.Entrance = start;
    var order = new List<Room> { start };

    // Grow a tree: attach new rooms to random existing ones, biased toward recent
rooms
    int guard = 0;
    while (map.Rooms.Count < trial.RoomCount && guard++ < 5000)
    {
        var from = order[Math.Max(0, order.Count - 1 - (int)
(Math.Pow(rng.NextDouble(), 2) * order.Count))];
        var dir = RoomLayout.AllDirs[rng.Next(4)];
        var step = RoomLayout.Step(dir);
        var p = new Point(from.Grid.X + step.X, from.Grid.Y + step.Y);
        if (Math.Abs(p.X) > GridRadius || Math.Abs(p.Y) > GridRadius ||
map.Rooms.ContainsKey(p)) continue;
        if (NeighbourCount(map, p) > 2) continue;          // avoid tight 2x2 clumps

        var room = new Room { Id = map.Rooms.Count, Grid = p, Type = RoomType.Normal
};
        map.Rooms[p] = room;
        Link(from, room, dir);
        order.Add(room);
    }

    // A few extra connections create loops (alternate routes).
    foreach (var r in map.Rooms.Values)
        foreach (var d in new[] { Dir.East, Dir.South })
        {
            var s = RoomLayout.Step(d);
            var n = map.At(new Point(r.Grid.X + s.X, r.Grid.Y + s.Y));
            if (n != null && !r.Links.ContainsKey(d) && rng.NextDouble() < 0.18)
Link(r, n, d);
        }

    map.ComputeDepths();
    map.AssignTypes(trial, rng, withNpcRoom, objectiveRooms);

    foreach (var r in map.Rooms.Values)
    {
        r.Template = rng.Next(RoomLayout.Templates.Length);
        r.Mirror = rng.Next(2) == 0;
```

```
        r.Layout = RoomLayout.Build(r.Template, r.Mirror, r.Links.Keys);   // doors =
keys of Links
    }
    return map;
}
```

Listing A.1 TrialMap.Generate (Trial/TrialMap.cs)

Appendix B
Data Structures in the Project

Table B.1 Fields in the code that are data structures

File	Name : type	Purpose
Core/Screen.cs	`_stack : List<Screen>`	Screen stack
Core/Assets.cs, Sprites.cs, Audio.cs	`Dictionary<string, ...>`, `Dictionary<(string,uint), SpriteSet>`	Image and sound caches
Trial/TrialMap.cs	`Rooms : Dictionary<Point, Room>`, `Queue<Room>`	Room graph and BFS
Trial/Room.cs	`Links : Dictionary<Dir, Room>`, `Pickups : List<LootPickup>`	Adjacency, loot
Trial/RoomLayout.cs	`Tiles : Tile[,]`, `Templates : string[][]`, `HashSet<Point>`	Room grid and flood fill
Trial/Pack.cs	`Members : List<Hunter>`, `Queue<Point>`	Chasing enemies and BFS
Trial/TrialRun.cs	`Party : List<PartyMember>`, `Flags : HashSet<string>`	State of one Trial
Combat/Battle.cs	`Party, Enemies : Combatant[4]`, `Fallen : List<Combatant>`	Formation
Combat/Combatant.cs	`Statuses : List<StatusInstance>`	Status effects
Combat/CombatAI.cs	`List<(T, double)>`	Weighted random choice
Data/ActionDef.cs	`Ranks` (`byte` bitmask)	Skill positions
Data/NpcDb.cs	`Nodes : Dictionary<string, DialogueNode>`	Dialogue graph
Meta/Profile.cs	`Dictionary<string, CharacterProgress>`, `List<string>`	Save file
Screens/CombatScreen.cs	`Queue<Combatant>`, `Queue<BattleEvent>`, `HashSet<Combatant>`	Turn queues, aiming

Appendix C
Sample Save File

A sample `profile.json` (the values are made up). The Dictionary becomes a JSON object keyed by id, and the List becomes a JSON array.

```
{
  "SecuredShards": 340,
  "Characters": {
    "relle":  { "HpLevel": 2, "DamageLevel": 1, "SpeedLevel": 0, "Weapon":
"rusted_blade", "Charm": null },
    "warden": { "HpLevel": 1, "DamageLevel": 0, "SpeedLevel": 0, "Weapon": null,
"Charm": "pale_locket" }
  },
  "Stash": [ "butchers_hatchet" ],
  "PartyOrder": [ "warden", "relle", "hunter", "priest" ],
  "Quests": { "maren_hunt": { "Progress": 8, "Claimed": true } },
  "Extractions": 4,
  "Defeats": 3
}
```

Listing C.1 Sample profile.json